



INTEGRATION REPORT · MAZJ TO REKAZ

Rekaz Merchant Public API: defect and gap report

54 findings from building a production booking integration. Most were measured by calling the live API with our own merchant credentials, and every such item says so on its Evidence line. Where a finding rests instead on the documented surface, on the dashboard, or on a question we asked in July 2026 and never had answered, the Evidence line says that instead. We have not asserted behaviour we did not observe.

Each finding carries an **expected behaviour** and an **acceptance criterion**, so any item can be picked up and closed without coming back to us for clarification.

Prepared by
MAZJ, Al Khobar
info@mazj.org

Date
28 July 2026

Tenant at time of testing
556 reservations, 100 subscriptions,
285 customers, 4 products, 1 branch

What this is, and why we wrote it

We built booking directly into our own website against this API rather than sending customers to the Rekaz storefront. That is a vote of confidence: the catalog, availability and reservation model is good enough to build on, and we would rather extend it than replace it.

Getting there took considerably longer than it should have. Six of the items below cost real debugging time, and three would have reached production as customer-visible failures if we had trusted the documentation. We are sending this because we think other merchants will hit the same ones, and because most are cheap to fix.

How to read this. Findings are grouped by kind and each carries a stable id (REK-001 and so on) that we will use in any follow-up. The two fields that matter most are **Expected behaviour** and **Acceptance criteria**: together they define done, precisely enough to be implemented and verified without a conversation.

SEVERITY	COUNT	MEANING
BLOCKER	7	Blocks a first-party booking experience, or loses money or data silently.
HIGH	17	Causes incorrect behaviour or a real security or compliance exposure.
MEDIUM	19	Costs integration time or forces a permanent workaround.
LOW	11	Worth fixing, low urgency.

If only four things change

1. **REK-001, the payment link.** It is returned relative while documented absolute. A merchant who follows the documentation sends every buyer to their own 404 on the last click, after the booking exists and is holding a room.
2. **REK-002, the ignored filters on GET /reservations .** Silent data loss that looks correct for months, then drops the long-lead bookings that matter most.
3. **REK-046, the forced identity binding.** A returning customer can only be booked by `customerId` , and there is no way to verify a phone number, so every merchant with a public booking form is forced into an impersonation vulnerability they cannot opt out of. We hit this directly.
4. **REK-026, marking an invoice paid externally.** The single highest-value change you could make for us. It would let merchants collect on their own domain through a Saudi gateway while Rekaz stays the invoicing and ZATCA authority.

A

Critical defects

Failures that cost money, lose data, or force a security exposure a merchant cannot opt out of. Most of them look correct from the outside.

REK-001 **BLOCKER** POST /reservations/bulk

paymentLink is returned as a relative path, contradicting the documentation

DOCUMENTED	Documentation shows an absolute URL: {"paymentLink": "https://platform.rekaz.io/i/NcRo"}
ACTUAL	Live API returned a relative path with a different shape: {"paymentLink": "/orders/pay/RMogH0PQc47FStqK"}
EVIDENCE	Observed on a real reservation created 2026-07-27. Verified that platform.rekaz.io/orders/pay/<id> answers 200 while mazj.sa/orders/pay/<id> answers 404, so the correct host cannot be inferred from the merchant's store domain.
IMPACT	A merchant who follows the documentation and redirects the browser to this value sends the buyer to their OWN origin and a 404. At that moment the reservation already exists in Pending state and is holding the room, the customer has not paid, and there is no documented way back to checkout (see REK-007). This is a silent, customer-facing revenue loss on the last click of every purchase.
EXPECTED BEHAVIOUR	Return a fully-qualified absolute URL, as documented.
ACCEPTANCE CRITERIA	For any successful reservation or subscription creation, paymentLink matches ^https:// and resolves to a working checkout page when opened with no base URL context.

A returning customer can only be booked by customerId, and combined with the lack of phone verification this FORCES an impersonation vulnerability on every merchant

DOCUMENTED	The documentation distinguishes the two (customerId for an existing customer, customerDetails for a new one) but nowhere states what happens when customerDetails carries a mobile number that already exists, nor that the answer is a hard 403 rather than a merge. A public booking form cannot know which case it is in without a separate lookup, and nothing warns that guessing wrong fails the write outright.
ACTUAL	They are not equivalent. customerDetails whose mobileNumber already belongs to a customer is rejected with HTTP 403 and the Arabic message رقم الجوال مسجلاً مسبقاً لعميل آخر ("the mobile number is already registered to another customer"). An unknown mobile is accepted. customerId is accepted.
EVIDENCE	Measured 2026-07-28 on both write endpoints with the same credential and headers, building the payload up one field at a time. The 403 appears exactly when customerDetails.mobileNumber is added and that number already exists. Rekaz neither deduplicates onto the existing record nor creates a second one: it refuses the write.
IMPACT	This is the most consequential item in this report after the payment boundary, because the two constraints compose into a vulnerability no merchant can avoid. A public booking form collects an unverified mobile number. If that number is already known, the ONLY payload Rekaz accepts is customerId, which silently binds the booking to that customer's account. So anyone who knows a member's mobile number can file real bookings and real invoices against that member. The merchant cannot opt out by sending the submitted details instead, because that path returns 403. And they cannot verify the number first, because no verification primitive exists (REK-029). We attempted exactly this fix on our own site on security grounds and had to withdraw it the same day: it broke booking for all 284 of our existing customers, on both flows.
EXPECTED BEHAVIOUR	Two changes, either of which resolves it. (1) Accept customerDetails for an existing mobile and attach the booking to that customer without overwriting their stored name or email, so a merchant can submit what the user typed without asserting an identity. (2) Provide a phone-verification primitive (REK-029) so a merchant can establish that the submitter holds the number before customerId is used. Separately, 403 is the wrong status: this is a business-rule conflict, not an authorization failure, and it should be 409 Conflict with a stable machine-readable code so it can be distinguished from a credential problem.
ACCEPTANCE CRITERIA	Either a booking submitted with customerDetails for an existing mobile succeeds and leaves the stored customer record unmodified, or a documented verification flow exists that a merchant can require before binding a booking to an existing customer. In both cases the current refusal returns 409 with a stable error code rather than 403.

REK-002 BLOCKER GET /reservations

Every documented filter is silently ignored

DOCUMENTED	<code>upcoming</code> , <code>dateMin</code> , <code>dateMax</code> and <code>statuses</code> are documented as filters.
ACTUAL	All four are ignored. The response is the unfiltered list, with no error, no warning, and nothing in the payload indicating the filter was not applied.
EVIDENCE	Measured against 555 live reservations. No filter: 100 rows, <code>totalCount</code> 555. <code>upcoming=true</code> : identical 100 rows. <code>dateMin + dateMax</code> over a 2-day window: identical 100 rows. <code>statuses=Confirmed</code> : identical 100 rows. Only <code>skipCount / maxResultCount</code> genuinely work.
IMPACT	This fails silently AND plausibly, which is the worst combination. A date-bounded query looks like it worked and returns whatever happened to be on page one. Combined with REK-003, an integration built the obvious way silently drops long-lead bookings.
EXPECTED BEHAVIOUR	Either implement the filters, or reject an unsupported filter with HTTP 400 naming the parameter. Silently ignoring a filter is strictly worse than refusing it, because the caller cannot detect it.
ACCEPTANCE CRITERIA	<code>GET /reservations?dateMin=X&dateMax=Y</code> returns only reservations whose <code>startAt</code> falls in [X,Y], and <code>totalCount</code> reflects the filtered count. If a filter is unsupported, the response is 400 with the offending parameter named in the error body.

REK-003 HIGH GET /reservations

Sort order is `creationTime DESC`, not `startAt`, and is undocumented

DOCUMENTED	No sort order is documented, and no <code>sort</code> parameter is offered.
ACTUAL	Rows are ordered by <code>creationTime</code> descending. Verified monotonically descending across all 555 rows, while <code>startAt</code> is not ordered at all.
EVIDENCE	Full pagination over the live tenant, comparing consecutive rows on both fields.
IMPACT	"Read page one, that is where the upcoming bookings are" is true only while customers book close to the date. A reservation created three months ago for next Tuesday sits on page four. Any integration built the obvious way silently drops exactly the long-lead bookings, which for an events hall are the most valuable ones. It looks correct for months and then loses the biggest reservation of the year.
EXPECTED BEHAVIOUR	Document the default sort order, and offer an explicit <code>sortBy</code> parameter supporting at least <code>startAt</code> and <code>creationTime</code> in both directions.
ACCEPTANCE CRITERIA	Documentation states the default order. <code>GET /reservations?sortBy=startAt&sortDirection=asc</code> returns rows ordered by <code>startAt</code> ascending.

REK-004 **HIGH** POST /reservations/bulk, POST /subscriptions

No idempotency keys on any write endpoint

DOCUMENTED	No idempotency mechanism is documented or offered.
ACTUAL	No <code>Idempotency-Key</code> header is accepted, no client-supplied reference field exists, and there is no way to query a write by request id. So a repeated write has nothing to deduplicate against.
EVIDENCE	Confirmed from the documented and live API surface. ⚠️ We have NOT deliberately created a duplicate booking on the production tenant to demonstrate it, because the only environment available is production (REK-030). The absence of the mechanism is the finding.
IMPACT	A double-tapped booking button, a browser retry on a flaky connection, or an impatient second tap while the first request is in flight each create a duplicate real booking and a duplicate invoice. The customer pays once and the merchant's operations team has a phantom reservation holding a room. This interacts badly with REK-042: the API is slow enough that timeouts are routine, and a timeout is exactly when a client retries.
EXPECTED BEHAVIOUR	Accept an <code>Idempotency-Key</code> request header on all write endpoints. A repeated key within a reasonable retention window returns the ORIGINAL response, including the original <code>paymentLink</code> , without creating a second record.
ACCEPTANCE CRITERIA	Two identical POSTs carrying the same <code>Idempotency-Key</code> produce exactly one reservation, and the second response is byte-identical to the first. A different body with the same key returns 422.

REK-005 **BLOCKER** Webhooks

Webhook payloads carry no signature of any kind

DOCUMENTED	21 events across four groups, 10 retries with exponential backoff. No signing mechanism is documented.
ACTUAL	No HMAC, no shared secret, no signing header, no timestamp binding.
EVIDENCE	Inspected the webhook registration surface at <code>platform.rekaz.io/identity/apikeys</code> and the documented payload envelope. ⚠️ We have NOT received a webhook: delivery needs a public URL and this site is not yet deployed. This finding rests on the absence of any documented signing mechanism, not on an inspected delivery.
IMPACT	Anyone who learns or guesses the webhook URL can forge a <code>ReservationConfirmedEvent</code> and tell a merchant's system that an unpaid booking is paid. For a merchant granting building access on booking confirmation, that is physical access granted on an unauthenticated HTTP request. The only safe mitigation available today is to treat every webhook as an untrusted hint and re-fetch the entity by id before acting, which is what we do, and which defeats most of the point of webhooks.
EXPECTED BEHAVIOUR	Sign the raw request body with HMAC-SHA256 keyed on a per-merchant secret shown in the dashboard. Include a timestamp inside the signed payload to prevent replay, and send both in a header. This is the standard pattern across Stripe, Tap, Moyasar and Paylink.
ACCEPTANCE CRITERIA	Every webhook delivery includes a signature header. A merchant can recompute it from the raw body plus their secret and reject a payload whose signature does not match or whose timestamp is outside an acceptable window. The verification procedure is documented with a worked example.

REK-006

HIGH

Reservation lifecycle

No documented expiry for unpaid Pending reservations, and no temporary hold

DOCUMENTED	Silent. The documentation does not state whether a Pending reservation holds its slot, whether unpaid reservations expire, or after how long.
ACTUAL	Unknown. We could not determine it without leaving a real unpaid booking in the production tenant for days, which we were not willing to do.
EVIDENCE	Raised as question 6 in our July 2026 review letter and never answered. Confirmed still undocumented as of 2026-07-28.
IMPACT	Because payment happens off-site (REK-025), there is always a window between reservation creation and payment. If Pending reservations hold the slot and never expire, abandoned checkouts accumulate and progressively block genuine availability, and a merchant has no way to distinguish an abandoned booking from a real one. If they do NOT hold the slot, two customers can pay for the same window. Both behaviours are defensible; not knowing which one applies makes it impossible to design the booking flow correctly.
EXPECTED BEHAVIOUR	Document the behaviour explicitly. If unpaid reservations expire, state the timeout and emit an event when it happens. Ideally, make the hold duration configurable per merchant.
ACCEPTANCE CRITERIA	Documentation states whether a Pending reservation reserves its slot and, if so, for how long. If auto-expiry exists, a corresponding webhook event fires and the slot becomes available again in <code>GET /reservations/slots</code> .

REK-007

HIGH

GET /reservations/{id}

A fetched reservation carries no payment link, so an interrupted checkout cannot be resumed

DOCUMENTED	Not addressed.
ACTUAL	<code>GET /reservations</code> exposes <code>orderId</code> , <code>orderPaymentStatusString</code> and <code>meetingUrl</code> , and nothing resembling a checkout URL.
EVIDENCE	Inspected the full response shape on the live tenant across 555 records.
IMPACT	If the merchant's process is interrupted between creating the reservation and handing the customer their payment link (a timeout, a closed tab, a crashed page), the link is unrecoverable. The booking exists, the customer owes money, and there is no programmatic way to send them to checkout. A human has to intervene on every occurrence. This is the direct cause of the worst branch in our own booking code, which has to tell the customer to contact us rather than retry.
EXPECTED BEHAVIOUR	Expose the payment link on the reservation and order resources, or provide <code>GET /orders/{id}/payment-link</code> that regenerates it for an unpaid order.
ACCEPTANCE CRITERIA	For any reservation whose payment status is unpaid, a documented endpoint returns a working absolute checkout URL.

HTTP 403 is returned for business-rule failures, making a real permission problem indistinguishable from a data conflict

DOCUMENTED	403 is not documented as a business-rule status.
ACTUAL	A booking rejected because the mobile number belongs to another customer returns 403, the same status the API returns when a request is genuinely refused (for example a rejected User-Agent, see REK-016).
EVIDENCE	Measured 2026-07-28. The business-rule 403 of REK-046 carries the application's JSON envelope with <code>code: null</code> . The 403 for a rejected User-Agent (REK-016) is not from the application at all: its body is the plain text <code>error code: 1010</code> , which is a Cloudflare WAF block. So the two are distinguishable by body, but only by knowing that in advance, and neither carries a machine-readable code.
IMPACT	Our client maps an unexpected 403 to an internal error, because a 403 from an API you are authenticated against normally means the credential is wrong. The result on our site was a booking failure that told the customer "an error occurred on our side, please try again", when the truthful message was "that number is already registered". Every merchant will make the same mapping, and every one of them will spend time debugging credentials for a data problem.
EXPECTED BEHAVIOUR	Use 409 for conflicts, 422 for validation failures, and reserve 403 for authorization. Populate the <code>code</code> field with a stable identifier so clients can branch on the reason rather than on a localised Arabic message.
ACCEPTANCE CRITERIA	A business-rule rejection returns a non-403 status and a non-null stable <code>error.code</code> that is documented.

Write endpoints return HTTP 500 for ordinary input errors instead of a 4xx, which makes a client bug indistinguishable from an outage

DOCUMENTED	Not addressed. The documentation describes 400 for validation failures.
ACTUAL	Five different malformed writes all returned HTTP 500 with <code>{"error":{"code":null,"message":"An internal error occurred during your request!"}}</code> . The same endpoint DOES produce correct statuses once a payload is well formed enough to reach domain logic: a real <code>customerId</code> with a nonexistent price returns 404 naming the entity. So the capability exists and the input-validation layer in front of it is what is missing.
EVIDENCE	Measured 2026-07-28 on <code>POST /reservations/bulk</code> . All five of these returned 500: an empty <code>items</code> array; a nonexistent <code>branchId</code> ; <code>branchId</code> omitted; <code>items</code> omitted; a nonexistent <code>priceId</code> with no customer supplied. The same call with a real <code>customerId</code> and a nonexistent price returned 404 with a specific message, proving the endpoint CAN produce a correct status once it gets that far.
IMPACT	This is expensive in a way that is easy to underestimate. A 500 is a server fault, so every reasonable client treats it as transient and retries. Our own client maps it to <code>upstream_unavailable</code> , which triggers the whole reconciliation path this API forces us to maintain (REK-004, REK-007): we hold the idempotency key, re-query to find out whether the booking landed, and if we cannot tell we close the key permanently and tell the customer to contact us. All of that machinery, and that customer-facing dead end, can be triggered by a request that was simply malformed and would never have succeeded. It also means a merchant cannot alert on 5xx as a health signal, because their own bugs are indistinguishable from your outages.
EXPECTED BEHAVIOUR	Validate write payloads and return 400 for a malformed body, 404 for a referenced entity that does not exist, and 409 for a business-rule conflict (see REK-047). Reserve 500 for genuine server faults.
ACCEPTANCE CRITERIA	A request with an empty <code>items</code> array, a missing required field, or an unparseable id returns 4xx naming the offending field. No client-side input error produces a 5xx.

REK-012 HIGH All endpoints

Two different error envelopes are returned depending on the failure class, and neither matches the documentation on its own

DOCUMENTED	One envelope is documented: <code>{"error": {"code", "message", "details", "validationErrors"}}.</code>
ACTUAL	Two shapes are in use, selected by where the failure happens. Model binding and query validation (HTTP 400) return RFC 9110 ProblemDetails : <code>{type, title, status, errors, traceId}.</code> Application and business failures (403, 404, 500) return the legacy documented envelope : <code>{error: {code, message, details, data, validationErrors}}.</code> ● In every legacy response observed, <code>error.code</code> was <code>null</code> .
EVIDENCE	Measured 2026-07-28 across six failure classes on the live tenant. 400 on a bad query parameter and 400 on a missing required parameter both returned ProblemDetails. 403 (business rule), 404 (unknown entity) and 500 (malformed body) all returned the legacy envelope with <code>code: null</code> . ⚠ This corrects an earlier version of this report, which stated that ProblemDetails had replaced the documented envelope; in fact both are live.
IMPACT	An integrator must implement two parsers, and cannot tell from the documentation that the second exists. Worse, the envelope that DOES carry a <code>code</code> field never populates it, so there is no stable machine-readable identifier for any error anywhere in the API. The only thing left to branch on is a localised human message (REK-018), which is precisely the thing that must never be branched on. This is the root cause of REK-047: we mapped an unexpected 403 to a generic internal error because nothing in the body distinguished a business-rule refusal from a credential problem.
EXPECTED BEHAVIOUR	Use one envelope. Whichever is chosen, populate a stable, documented, non-null machine-readable <code>code</code> on every error, and document the full list of codes.
ACCEPTANCE CRITERIA	Every error response, at every status, carries the same top-level shape and a non-null <code>code</code> drawn from a documented enumeration. A client can branch on <code>code</code> alone without inspecting the message or the status.

REK-008 MEDIUM GET /reservations, GET /customers

Mobile numbers are stored and returned without the leading plus, silently breaking comparisons

DOCUMENTED	Not documented in either direction.
ACTUAL	We send <code>+966500000000</code> in <code>customerDetails.mobileNumber</code> . The API returns <code>966500000000</code> .
EVIDENCE	Compared submitted values against returned values on records we created ourselves.
IMPACT	A direct string comparison never matches, so any reconciliation, deduplication or customer-lookup logic silently and permanently reports "not found". It fails in the safe-looking direction, which is why it survives review: nothing errors, the answer is just always no.
EXPECTED BEHAVIOUR	Return numbers in the same canonical form they were submitted in, ideally E.164 with the leading plus. If normalisation is intentional, document it prominently.
ACCEPTANCE CRITERIA	A number submitted as <code>+966500000000</code> is returned in a documented canonical form, and the documentation states what that form is.

B

Documentation contradicts the API

Each of these cost us time. Most are a documentation fix rather than a code change; REK-012 is the exception and needs both.

REK-009 **MEDIUM** Authentication

Tenant header is documented with one underscore; the API requires two

DOCUMENTED	The Quick Start page documents <code>_tenant</code> .
ACTUAL	<code>__tenant</code> (two underscores) is required. One underscore does not authenticate.
EVIDENCE	Both forms sent against the live API.
IMPACT	A new integrator's first request fails with an authentication error that points them at their credentials rather than at a header name. Cost us time on day one.
EXPECTED BEHAVIOUR	Correct the Quick Start page.
ACCEPTANCE CRITERIA	Every documentation page shows <code>__tenant</code> .

REK-010 **MEDIUM** Authentication

Two documentation pages describe two different authentication models

DOCUMENTED	Quick Start describes a token valid for one hour requiring refresh. The API page describes HTTP Basic with a non-expiring key and secret.
ACTUAL	HTTP Basic with a non-expiring key pair is what the API accepts. There is no refresh flow.
EVIDENCE	Basic auth with the issued key pair worked continuously across a full day of testing with no refresh.
IMPACT	An integrator implements token refresh machinery that does not exist and cannot be tested, then removes it. Pure wasted effort.
EXPECTED BEHAVIOUR	Retire or correct the Quick Start page.
ACCEPTANCE CRITERIA	Only one authentication model is described across the documentation.

REK-011 **MEDIUM** GET /reservations/slots

MinQuantity is documented as optional but is required

DOCUMENTED	Listed as optional.
ACTUAL	Omitting it returns 400 with the Arabic message <code>بين 1 و MinQuantity يجب أن يكون الحقل 2147483647</code> .
EVIDENCE	Request sent without the parameter against the live API.
IMPACT	The single most important endpoint for building a booking UI fails on the first call, with an Arabic error, for a parameter the documentation says is optional.
EXPECTED BEHAVIOUR	Mark it required, or apply a server-side default of 1.
ACCEPTANCE CRITERIA	Either the documentation marks <code>MinQuantity</code> required, or omitting it succeeds and behaves as <code>MinQuantity=1</code> .

REK-013 LOW POST /reservations/bulk

invoiceld is documented on the booking response but came back undefined

DOCUMENTED	Documented as present.
ACTUAL	Undefined on a real reservation creation.
EVIDENCE	Observed on the live booking created 2026-07-27.
IMPACT	A merchant using invoiceld as their operational pointer to a booking has no pointer. We had to log the entire response and key on reservationNumber instead.
EXPECTED BEHAVIOUR	Either populate it or remove it from the documented response.
ACCEPTANCE CRITERIA	invoiceld is present on every successful creation response, or absent from the documentation.

REK-014 LOW GET /reservations/slots

The documented slot example shows a state that does not occur

DOCUMENTED	Example shows isAvailable: false alongside availableReservationsCount: 1.
ACTUAL	Across 192 real slots over a three-week window, zero had isAvailable: false.
EVIDENCE	Full slot enumeration for both reservation products over three consecutive weeks.
IMPACT	We wrote defensive code for a contradictory state that appears not to exist, and spent time trying to work out what it meant. If the example is simply wrong, it should be corrected; if the state IS reachable, the condition that produces it needs documenting.
EXPECTED BEHAVIOUR	Correct the example, or document the conditions under which the two fields disagree.
ACCEPTANCE CRITERIA	The documented example reflects a state the API actually produces.



Undocumented behaviour

The API does something reasonable that nobody could have predicted from the documentation.

REK-015 **HIGH** GET /reservations/slots

The requested date range is padded rather than honoured

DOCUMENTED	Not documented.
ACTUAL	Requesting <code>StartDate=2026-07-28&EndDate=2026-07-29</code> returned slots for the 27th, 28th AND 29th. Requesting a Friday-only range returned the preceding Thursday.
EVIDENCE	Multiple range requests compared against the returned <code>from</code> dates.
IMPACT	A date picker that renders the response directly offers customers slots on days they did not ask about, and on a Friday offers them Thursday while labelling it Friday. Every caller must re-filter to the range it requested and separately drop <code>isOutDated: true</code> entries. Neither requirement is documented, so the naive implementation is wrong and looks right.
EXPECTED BEHAVIOUR	Honour the requested range exactly. If padding is intentional (for example to include a window that starts before the range), document it and explain the rule.
ACCEPTANCE CRITERIA	Every slot returned has its <code>from</code> within the requested range, or the padding rule is documented precisely enough to filter against.

REK-050 **MEDIUM** All list endpoints

Three different collection envelopes are returned by different endpoints, so a generic client cannot be written

DOCUMENTED	Collections are described uniformly.
ACTUAL	Three shapes. <code>/branches</code> returns a bare JSON array. <code>/providers</code> returns <code>{items}</code> with NO <code>totalCount</code> . Everything else returns the full <code>{items, totalCount}</code> envelope.
EVIDENCE	Measured against the live tenant and pinned by a regression test in our own suite, which asserts that <code>/providers</code> has no <code>totalCount</code> property so that we notice if it ever gains one.
IMPACT	A generic paging helper cannot be written. Assuming the envelope on <code>/branches</code> yields <code>undefined</code> rather than an error, so the bug surfaces as an empty list rather than a failure, which is the hardest kind to notice. We carry a separate typed shape per endpoint as a result.
EXPECTED BEHAVIOUR	Return one envelope shape from every collection endpoint, including <code>totalCount</code> .
ACCEPTANCE CRITERIA	Every list endpoint returns <code>{items, totalCount}</code> , and a single client-side paging helper works against all of them unchanged.

REK-054 LOW GET /reservations/slots

A closed day and a fully booked day are indistinguishable: both are simply absent

DOCUMENTED	Not addressed.
ACTUAL	Days on which the venue does not open return no slots at all, exactly like a day whose slots are all taken. Fridays and Saturdays return nothing for our tenant.
EVIDENCE	Verified across three consecutive weeks; the closed days are absent from the response entirely rather than present and unavailable.
IMPACT	A booking UI cannot tell a visitor "we are closed on Fridays" rather than "Friday is fully booked", which are very different messages and lead to different behaviour. We hardcode the closed days in our own front end as a result, which will silently go wrong the day opening hours change in the Rekaz dashboard.
EXPECTED BEHAVIOUR	Return closed days explicitly (an empty day marked closed, or a separate opening-hours resource), so a client can distinguish unavailable from not offered.
ACCEPTANCE CRITERIA	A client can tell, from the API alone, whether a day has no slots because the venue is closed or because they are all taken.

REK-016 MEDIUM All endpoints

Requests are rejected with 403 based on User-Agent

DOCUMENTED	Not documented.
ACTUAL	<code>Python-urllib/3.12</code> receives 403 Forbidden, as does a request with no User-Agent at all. <code>curl/8.7.1</code> , <code>Mozilla/5.0</code> , <code>node</code> and our own agent all receive 200. 🚫 The refusal is NOT from your application: the body is the plain text <code>error code: 1010</code> , which is a Cloudflare WAF block on browser signature. It never reaches Rekaz code, so it will not appear in your application logs.
EVIDENCE	Identical authenticated requests differing only in User-Agent, 2026-07-27 and re-confirmed 2026-07-28. The 403 body was captured on the second pass and is <code>error code: 1010</code> in plain text, not JSON.
IMPACT	The 403 is indistinguishable from an authentication failure, so an integrator debugs their credentials instead of their headers. Worse, this is a latent production outage for anyone relying on a client library's default agent: a runtime upgrade that changes the default breaks a working integration with no code change on the merchant side, at whatever hour the deploy happens.
EXPECTED BEHAVIOUR	Allow-list legitimate API clients at the edge, or exempt the API hostname from the browser-signature rule entirely. An API is not a website and its clients are not browsers. If the rule must stay, document it and return a body that says so.
ACCEPTANCE CRITERIA	A request from a default HTTP client (for example Python's <code>urllib</code>) with valid credentials receives an application response, not a Cloudflare block. If the rule is retained, the documentation names it and the response identifies itself as an edge block rather than an API refusal.

REK-017 **MEDIUM** GET /reservations/slots

basePrice and priceWithTax are always zero

DOCUMENTED	Present in the documented response with no note.
ACTUAL	Both fields were 0 in every observed slot, while totalPrice and totalAfterDiscount carried correct values.
EVIDENCE	192 slots across two products.
IMPACT	A merchant computing VAT from priceWithTax bills zero tax. In Saudi Arabia that is a ZATCA compliance problem, not a display bug.
EXPECTED BEHAVIOUR	Populate them, or remove them from the response and the documentation.
ACCEPTANCE CRITERIA	Either both fields carry correct values, or neither appears in the response.

REK-018 **MEDIUM** All endpoints

Error message localisation is inconsistent: some strings honour Accept-Language, others are hardcoded English, and nothing says which

DOCUMENTED	No language behaviour is documented.
ACTUAL	Accept-Language IS honoured, but only for some strings. Measured on the same request with only the header changed: the legacy envelope's message localises correctly (en gives "Mobile number already exists with different customer", ar gives the Arabic). Inside a ProblemDetails body, title is always English, a range violation localises (MinQuantity), and a type violation does not (PriceId stays English under Accept-Language: ar).
EVIDENCE	Measured 2026-07-28 across four failure classes with en and ar. ⚠️ This CORRECTS an earlier version of this report, and our own client, both of which asserted that messages were Arabic regardless of the header. They are not; we had simply never sent it.
IMPACT	Smaller than the original claim, and still real. A merchant cannot tell which strings will localise, so the safe assumption is that none will, and every integrator builds a full error-code-to-copy mapping layer anyway. The absence of a stable machine-readable code (see REK-012) is what makes that layer necessary, because the only thing left to branch on is a message whose language is not guaranteed.
EXPECTED BEHAVIOUR	Localise consistently: every human-readable string in an error response should honour Accept-Language, or none should and the documentation should say so. Document the supported languages and the default.
ACCEPTANCE CRITERIA	For a given failing request, every human-readable string in the response body is in the requested language, and the documentation states which languages are supported.

REK-019 **HIGH** All endpoints

Error bodies echo submitted values verbatim, which is a personal-data exposure for merchants

DOCUMENTED	Not addressed.
ACTUAL	Validation errors echo the submitted value, for example <code>{"errors": {"PriceId": ["The value 'BAD' is not valid for PriceId."]}}</code> .
EVIDENCE	Captured 400 response from the live API.
IMPACT	Booking payloads carry <code>customerDetails</code> with the customer's name, mobile number and email. A validation failure on any of those fields therefore echoes that personal data into the response body. Any merchant who logs raw upstream responses (which is the normal, recommended practice for debugging a third-party API) is now writing customer personal data into their log store, which under Saudi PDPL is a secondary copy that can turn a lawful collection into an unlawful one. We discovered this in our own code and had to redact it. Most merchants will not notice.
EXPECTED BEHAVIOUR	Do not echo submitted values for fields that can carry personal data. Return the field name and a generic reason.
ACCEPTANCE CRITERIA	A validation failure on <code>customerDetails.mobileNumber</code> names the field but does not include the submitted number in the response body.

REK-020 **MEDIUM** All list endpoints

Filter support is inconsistent across endpoints with nothing to distinguish them

DOCUMENTED	Filters are documented uniformly, as though they all work.
ACTUAL	<code>GET /customers?mobileNumber=</code> filters correctly (284 customers narrowed to 1). <code>customerId</code> genuinely filters <code>GET /subscriptions</code> (98 rows to 1). Every filter on <code>GET /reservations</code> is ignored (REK-002).
EVIDENCE	Each filter tested independently against the live tenant.
IMPACT	A merchant has no way to know which filters are real without testing every one against production data. Any filter they trust without testing is a silent correctness bug.
EXPECTED BEHAVIOUR	Make filter support consistent, or mark unsupported filters clearly in the documentation per endpoint.
ACCEPTANCE CRITERIA	For every documented filter on every list endpoint, either it demonstrably filters or it is documented as unsupported.

REK-021 **MEDIUM** GET /products

Price ids rotate when a price is edited, which is not documented

DOCUMENTED	Not documented.
ACTUAL	Every <code>pricing[].id</code> differs from its <code>immutableId</code> . The live price ids carry a July 2026 prefix while the immutable ids date to 2024.
EVIDENCE	Compared id and immutableId across all four products on the live tenant.
IMPACT	A page rendered before a price edit submits an id that no longer exists. The behaviour is reasonable but the consequence is severe and entirely undocumented: an integrator who caches or hardcodes a price id has a booking flow that breaks the next time someone edits a price in the dashboard, with no warning.
EXPECTED BEHAVIOUR	Document that <code>id</code> is volatile and <code>immutableId</code> is the stable handle, and state it prominently wherever price ids appear.
ACCEPTANCE CRITERIA	The documentation for the product resource explains the difference and instructs integrators to key on <code>immutableId</code> .

REK-022 **LOW** GET /reservations

Cancelled reservations remain in the default list response

DOCUMENTED	Not stated.
ACTUAL	Cancelled reservations are returned with <code>status: "Cancelled"</code> and no filter excludes them (see REK-002).
EVIDENCE	Observed after cancelling our own test reservation.
IMPACT	Any count of bookings is wrong unless the caller filters manually. Combined with the ignored <code>statuses</code> filter, the caller must fetch everything and filter in code.
EXPECTED BEHAVIOUR	Document the behaviour, and make the <code>statuses</code> filter work so callers can exclude them server-side.
ACCEPTANCE CRITERIA	<code>GET /reservations?statuses=Confirmed</code> excludes cancelled records.

REK-023 **LOW** GET /reservations

customerMobile and customerName come back as empty strings on some records

DOCUMENTED	Typed as strings with no nullability note.
ACTUAL	Empty strings on some older records. Populated on records created through the API with <code>customerDetails</code> .
EVIDENCE	Observed across the live tenant's 555 reservations.
IMPACT	Neither field can be a required match key for reconciliation or display. A UI rendering them directly shows an unexplained blank cell that reads as a layout bug.
EXPECTED BEHAVIOUR	Return null rather than an empty string when the value is genuinely absent, and document which records can lack it.
ACCEPTANCE CRITERIA	Absent values are null, and the documentation notes that historical records may lack customer identity fields.

Webhook payloads use PascalCase while every REST response uses camelCase

DOCUMENTED	Examples are inconsistent with each other: some show PascalCase with a Data wrapper, some show camelCase without.
ACTUAL	The documented envelope is { Id, EventName, CreatedAt, Data } in PascalCase, while every REST response is camelCase. ⚠ Documented, not observed: we have not yet received a delivery (see REK-005).
EVIDENCE	Webhook registration surface and documented examples.
IMPACT	Two deserialisation conventions in one integration, and the documented examples do not agree with each other, so an integrator cannot tell which is current.
EXPECTED BEHAVIOUR	Use one casing convention across REST and webhooks, or state the rule explicitly and make every example consistent with it.
ACCEPTANCE CRITERIA	All documented webhook examples use the same casing as the delivered payload.



Missing capabilities that block a first-party experience

These are the reasons we cannot finish what we set out to build.

REK-025 **BLOCKER** Payments

No payments API, so card entry cannot happen on the merchant's own domain

DOCUMENTED	No payments API is offered.
ACTUAL	The only path to payment is the Rekaz-hosted checkout reached via <code>paymentLink</code> .
EVIDENCE	Full surface review of the documented and live endpoints.
IMPACT	This is the single hard architectural constraint on building a first-party booking experience. We built our entire booking flow on our own domain and still have to hand the customer off to a third-party page for the final step, which is where checkout abandonment concentrates. It is also what makes REK-001 so damaging: the one step we cannot control is the one whose link shape is wrong.
EXPECTED BEHAVIOUR	Either a payments API, or the ability to mark an invoice paid externally (REK-026).
ACCEPTANCE CRITERIA	A merchant can collect payment on their own domain and have Rekaz reflect the paid state.

REK-026 **BLOCKER** Payments, invoicing

No way to mark an invoice paid from outside Rekaz, so a merchant cannot use their own payment gateway

DOCUMENTED	Not offered.
ACTUAL	No endpoint accepts an external payment confirmation.
EVIDENCE	Raised as question 2 in our July 2026 review letter and never answered.
IMPACT	This is the highest-value single change Rekaz could make for us, and we believe for most merchants of our size. We want to collect payment on our own domain through a Saudi gateway (Mada, Moyasar, Tap or HyperPay) while Rekaz remains the system of record for the invoice and the ZATCA e-invoicing obligation. That combination is currently impossible: either we use Rekaz's hosted page and lose control of the last click, or we take payment ourselves and lose Rekaz as the invoicing authority, which we are not willing to do. There is no third option.
EXPECTED BEHAVIOUR	An authenticated endpoint that marks an order paid, carrying an external payment reference, gateway name and amount, and which triggers the same downstream state changes and webhooks as a Rekaz-collected payment.
ACCEPTANCE CRITERIA	A merchant can collect payment through their own gateway, call the endpoint, and see the reservation move to Confirmed with a correct invoice issued by Rekaz.

REK-049 **HIGH** Hosted checkout

The hosted checkout accepts no return URL, so a paying customer never comes back to the merchant's site

DOCUMENTED	No return, callback or redirect parameter is documented on the booking creation call.
ACTUAL	We found no way to tell Rekaz where to send the buyer after payment. The journey ends on the Rekaz-hosted page.
EVIDENCE	Documented and live surface review of <code>POST /reservations/bulk</code> and <code>POST /subscriptions</code> ; no such field exists in the request, and the response carries only <code>paymentLink</code> .
IMPACT	A merchant who has built booking on their own domain (which is what we did) loses the customer at the last step. There is no confirmation page in the merchant's own brand, no opportunity to show what was booked or what happens next, and no reliable client-side signal that payment completed, so the merchant learns the outcome only from a webhook (unsigned, REK-005) or by polling. For a coworking space, the moment after payment is when a first-time customer most needs directions, access instructions and a welcome. We cannot give them one.
EXPECTED BEHAVIOUR	Accept a <code>returnUrl</code> (and ideally a <code>cancelUrl</code>) on the booking creation call, and redirect the buyer there after the payment attempt with at least the order reference and outcome as query parameters.
ACCEPTANCE CRITERIA	A booking created with a <code>returnUrl</code> sends the buyer back to that URL after payment, carrying enough context for the merchant to render a confirmation without a second API call.

REK-027 **HIGH** Hosted checkout

The hosted checkout page cannot be branded or served from the merchant's domain

DOCUMENTED	Not addressed.
ACTUAL	No branding configuration and no custom-domain option are documented or visible in the dashboard.
EVIDENCE	Raised as question 1 in our July 2026 review letter and never answered.
IMPACT	The last step of every purchase moves the buyer to a page that does not look like the merchant's brand and sits on a domain they do not recognise. For a customer paying for a private office, that is exactly the moment trust matters most. This is a lesser version of REK-026 and would be a meaningful improvement even without a payments API.
EXPECTED BEHAVIOUR	Allow a merchant logo, colour and a custom subdomain (for example <code>pay.mazj.org</code> via CNAME) on the hosted checkout.
ACCEPTANCE CRITERIA	A merchant can configure branding and a custom domain, and the checkout page reflects both.

REK-028 HIGH Customer identity

No end-customer authentication of any kind

DOCUMENTED	Not offered.
ACTUAL	No customer login, no OTP, no per-customer scoped token.
EVIDENCE	Full surface review.
IMPACT	A "my bookings" page means building authentication from scratch and then querying with an admin-scope key, where one missing filter exposes the entire customer base (see REK-044). Every merchant who wants this feature must independently build the same risky thing.
EXPECTED BEHAVIOUR	A customer-scoped token, obtainable by proving control of the registered mobile number or email, that can only read that customer's own records.
ACCEPTANCE CRITERIA	A merchant can obtain a token scoped to one customer and use it to list only that customer's reservations and subscriptions.

REK-029 HIGH Customer identity

No phone-verification primitive, so a phone number cannot be treated as identity

DOCUMENTED	Not offered.
ACTUAL	No OTP send or verify endpoint exists.
EVIDENCE	Full surface review.
IMPACT	This has a direct security consequence we hit ourselves, and the API leaves no way out of it. A public booking form collects a mobile number, and for a customer Rekaz already knows, binding the booking to their <code>customerId</code> is the ONLY payload Rekaz accepts (REK-046). With no way to prove the submitter holds that number, that binding lets anyone who knows a member's mobile file real bookings and real invoices against their account. We tried to close it by always sending the submitted details instead, which is the only mitigation available without a verification primitive. Rekaz refuses that payload with 403, so we had to withdraw the fix the same day. A merchant on this API cannot even choose to create a duplicate record rather than assert an identity they have not verified.
EXPECTED BEHAVIOUR	An OTP send and verify pair, returning a short-lived proof token that can be attached to a booking to establish that the submitter controls the number.
ACCEPTANCE CRITERIA	A merchant can verify a number and attach the resulting proof to a reservation, and Rekaz records the booking as verified.

No sandbox environment, so all development happens against production data

DOCUMENTED	No sandbox is mentioned.
ACTUAL	None exists.
EVIDENCE	Raised as question 3 in our July 2026 review letter and never answered.
IMPACT	Every integration test runs against real customer data. We deliberately kept our automated test suite read-only for this reason, and the one write test we needed was run by hand once against production and then deleted. That means no merchant can run write-path tests in CI, so the booking flow (the part that takes money) is the least testable part of the integration. Test bookings also appear in the operations dashboard and distort reporting.
EXPECTED BEHAVIOUR	A sandbox tenant with separate credentials, seeded catalog data, and a simulated payment step that can be driven to both success and failure.
ACCEPTANCE CRITERIA	A merchant can create, pay and cancel reservations in a sandbox with no effect on production data or reporting.

File-type custom fields have no upload endpoint

DOCUMENTED	Field type 10 is documented as a file field.
ACTUAL	No upload endpoint is documented or discoverable.
EVIDENCE	Our events hall product carries a file field (السجل التجاري - تصريح النشاط). We could not implement it and shipped the booking flow without it.
IMPACT	A merchant can configure a field their customers cannot fill through the API. For us this is a commercial-registration document for event bookings, which is exactly the kind of thing a venue needs before confirming a large booking.
EXPECTED BEHAVIOUR	A documented upload endpoint returning a reference that can be submitted as the field's value.
ACCEPTANCE CRITERIA	A merchant can upload a file, receive a reference, and submit it as a type-10 custom field on a reservation.



Missing capabilities that degrade one

Workable without, but each forces a compromise.

REK-032 **MEDIUM** Developer experience

No OpenAPI specification and no SDKs

DOCUMENTED	Not offered.
ACTUAL	None available.
EVIDENCE	Raised as question 11 in our July 2026 review letter and never answered.
IMPACT	Every type in our integration is hand-written and must be re-reviewed whenever Rekaz ships a change. There is no drift alarm, so a breaking change is discovered by a customer rather than by a build. This also makes every other item in this report more expensive to work around, because there is no generated client to update in one place.
EXPECTED BEHAVIOUR	Publish an OpenAPI 3.x specification covering every public endpoint, kept in sync with the implementation.
ACCEPTANCE CRITERIA	A merchant can generate a typed client from the specification and it compiles against the live API.

REK-051 **MEDIUM** GET /products

No endpoint accepts immutableId as input, so resolving a stable price handle means downloading the whole catalog

DOCUMENTED	<code>immutableId</code> is exposed on every price but no endpoint documents it as a lookup key.
ACTUAL	There is no <code>GET /prices/{immutableId}</code> and no filter accepting one. The only way to turn an <code>immutableId</code> into the current <code>id</code> is to fetch <code>GET /products</code> and scan every product's pricing array.
EVIDENCE	Our booking service does exactly that on every booking attempt, because REK-021 makes <code>id</code> unsafe to cache.
IMPACT	The advice implied by REK-021 (key on the immutable handle) is sound and the API does not support acting on it. Every booking pays for a full catalog download against an endpoint measured between 1.2s and 10.8s (REK-042), on the critical path, before anything else can happen. That single call is the largest fixed cost in our booking flow and it exists only to work around a rotating identifier.
EXPECTED BEHAVIOUR	Accept <code>immutableId</code> wherever <code>priceId</code> is accepted, or provide a lookup endpoint that resolves one to the other.
ACCEPTANCE CRITERIA	A merchant can resolve or book against an <code>immutableId</code> without fetching the full product catalog.

REK-052 **MEDIUM** GET /reservations/slots

Availability can only be queried one price at a time, so a product with several durations needs several slow calls

DOCUMENTED	<code>PriceId</code> is a single required value.
ACTUAL	Exactly one price per request. A product offering four durations has four separate calendars and needs four requests to show them.
EVIDENCE	The endpoint accepts a single <code>PriceId</code> ; our booking UI refetches availability every time the visitor changes duration.
IMPACT	On an API measured up to 10.8s (REK-042), where concurrency degrades badly (REK-043), this forces a serial request every time a visitor changes their mind about duration. It is the reason our date picker cannot show a combined calendar, and the reason changing duration is visibly slow. Since the same API serves the storefront where real customers check out, we cannot compensate by parallelising.
EXPECTED BEHAVIOUR	Accept multiple price ids per request and return availability keyed by price, or accept a product id and return every price's availability in one response.
ACCEPTANCE CRITERIA	A single request returns availability for all prices of one product.

REK-053 **MEDIUM** GET /reservations

reservationNumber, the only human-facing reference, cannot be used to fetch or filter a reservation

DOCUMENTED	<code>reservationNumber</code> is returned on every reservation. No endpoint accepts it.
ACTUAL	GET /reservations/{id} requires the GUID. There is no lookup by reservation number and, since the documented filters are ignored (REK-002), no way to search for one either.
EVIDENCE	Live surface review; the GUID is the only accepted identifier.
IMPACT	This is the number a customer reads out on the phone, and it is the one identifier a merchant cannot use to find their booking. Our own worst-case path hands the customer a <code>reservationNumber</code> and asks them to contact us, at which point staff must page through reservations by hand to find it. The GUID, meanwhile, is known only from a creation response that in that exact scenario we never received (REK-007).
EXPECTED BEHAVIOUR	Accept <code>reservationNumber</code> as a lookup key, or make it a working filter on the list endpoint.
ACCEPTANCE CRITERIA	A merchant can retrieve a reservation using only the number the customer can read to them.

REK-033 **LOW** Discounts**No coupon validation endpoint**

DOCUMENTED	A direct discount (Fixed or Percentage) can be passed, but there is no way to validate a customer-entered code.
ACTUAL	Validation is the merchant's responsibility.
EVIDENCE	Documented surface review.
IMPACT	Discount logic ends up split across two systems, with the merchant holding the half that decides whether a code is valid and Rekaz holding the half that applies it. The two will drift.
EXPECTED BEHAVIOUR	An endpoint that validates a coupon code and returns its resolved discount.
ACCEPTANCE CRITERIA	A merchant can submit a code and receive either the discount it resolves to or a documented rejection reason.

REK-034 **MEDIUM** Invoicing**No invoice retrieval endpoint, despite invoice fields appearing on subscriptions**

DOCUMENTED	<code>lastInvoiceCode</code> and <code>lastInvoiceStatus</code> appear in subscription data, but no endpoint retrieves the invoice itself.
ACTUAL	No invoice or e-invoicing endpoint exists.
EVIDENCE	Documented and live surface review.
IMPACT	A merchant can see that an invoice exists and what its status is, but cannot fetch it, send it, or reconcile it. For ZATCA e-invoicing this matters: the merchant is accountable for records they cannot retrieve programmatically.
EXPECTED BEHAVIOUR	An endpoint returning the invoice, including the ZATCA e-invoicing payload or a link to it.
ACCEPTANCE CRITERIA	A merchant can retrieve an invoice by code and obtain the compliant document.

REK-035 **LOW** Refunds**No refund endpoint**

DOCUMENTED	Not offered.
ACTUAL	Refunds appear to be dashboard-only.
EVIDENCE	Raised as question 10 in our July 2026 review letter and never answered.
IMPACT	Any refund flow requires a human in the Rekaz dashboard, so a merchant cannot offer self-service cancellation with refund.
EXPECTED BEHAVIOUR	A refund endpoint, or explicit confirmation that refunds are deliberately dashboard-only.
ACCEPTANCE CRITERIA	Either a documented refund endpoint exists, or the documentation states that refunds are manual.

REK-036

LOW

GET /products

No product images in the API

DOCUMENTED	<code>productProviders[].image</code> exists in the schema.
ACTUAL	Null on every product on our tenant, and no image gallery is exposed.
EVIDENCE	Live catalog inspection.
IMPACT	Space photography stays in the merchant's repository and drifts from the Rekaz catalog. A merchant updating a room's photo in Rekaz sees no change on their own site, and vice versa.
EXPECTED BEHAVIOUR	Expose product images through the API, or document that the field is unused.
ACCEPTANCE CRITERIA	Either images are returned, or the field is removed from the documented schema.

REK-037

LOW

All list endpoints

Hard 100-record page cap with no cursor pagination

DOCUMENTED	Documented as <code>skipCount</code> and <code>maxResultCount</code> .
ACTUAL	Silently capped at 100. <code>maxResultCount=101</code> , <code>250</code> and <code>1000</code> all return exactly 100 rows with HTTP 200 and no indication the request was reduced. <code>skipCount</code> works correctly and pages are disjoint.
EVIDENCE	Measured 2026-07-28 across <code>maxResultCount</code> 1, 50, 100, 101, 250 and 1000 against 556 live reservations, and <code>skipCount</code> paging verified disjoint.
IMPACT	Combined with REK-002 (no working filters) and REK-042 (up to 10.8s per request), assembling a complete picture of 555 reservations takes six sequential requests and can take most of a minute. This is the direct cause of our having to cache aggressively.
EXPECTED BEHAVIOUR	Raise the cap, or offer cursor-based pagination, or (best) make the filters work so full enumeration is unnecessary.
ACCEPTANCE CRITERIA	A merchant can retrieve a filtered result set without enumerating the entire collection.

REK-038

MEDIUM

All endpoints

A `RateLimit` error code exists but no rate limits are documented

DOCUMENTED	A <code>RateLimit</code> code appears in the error list. No limits, headers or retry guidance are documented.
ACTUAL	No 429 response, no <code>Retry-After</code> header, and no documented threshold were observed, including during the concurrency collapse described in REK-043.
EVIDENCE	Raised as question 7 in our July 2026 review letter and never answered.
IMPACT	A merchant cannot distinguish "you are being throttled" from "the API is down", so correct backoff is impossible to implement. We serialise every request and cache for 60 seconds purely because we cannot tell the difference.
EXPECTED BEHAVIOUR	Document the limits, return 429 with <code>Retry-After</code> when they are hit, and ideally expose remaining quota in response headers.
ACCEPTANCE CRITERIA	Exceeding the limit returns 429 with <code>Retry-After</code> , and the limit is documented.



Data quality and internationalisation

Where the API's bilingual promise does not hold.

REK-039 **HIGH** GET /products, GET /branches

nameEn is byte-identical to nameAr, so there is no English content

DOCUMENTED	Bilingual catalog data is presented as an advantage of the API.
ACTUAL	<code>nameEn</code> returns Arabic on every product and every branch on our tenant.
EVIDENCE	Byte comparison of nameEn and nameAr across the full catalog.
IMPACT	A bilingual merchant cannot use Rekaz as the source of truth for English content and must maintain a parallel translation layer keyed by product id, which then silently drifts when someone edits a product in the dashboard. Rendering <code>nameEn</code> on an English page emits Arabic. This is the advantage the documentation advertises, and for us it does not exist.
EXPECTED BEHAVIOUR	Provide real English fields in the dashboard. At minimum, document that <code>nameEn</code> falls back to the Arabic value when unset, so integrators do not assume it is populated.
ACCEPTANCE CRITERIA	Either English content can be entered per product and is returned in nameEn, or the fallback behaviour is documented.

REK-040 **MEDIUM** Custom fields

Custom field labels and placeholders are Arabic only

DOCUMENTED	A localizedLabel structure exists with an OtherLanguages map.
ACTUAL	<code>localizedLabel.OtherLanguages.en</code> is an empty string on every field.
EVIDENCE	Inspected all three custom fields on the events hall product.
IMPACT	Same as REK-039, applied to form fields. An English booking form must hold its own copy of every label, keyed by field id, and those keys are GUIDs.
EXPECTED BEHAVIOUR	Allow English labels to be entered and return them.
ACCEPTANCE CRITERIA	A merchant can enter an English label in the dashboard and receive it in OtherLanguages.en.

REK-041 **LOW** GET /products

A product can have an empty branchIds array while others are populated

DOCUMENTED	Not addressed.
ACTUAL	Our events hall product returns <code>branchIds: []</code> while the other three name the branch.
EVIDENCE	Live catalog inspection.
IMPACT	Any integration filtering products by branch silently drops that product from the catalog. We suspect a data issue on our tenant rather than an API defect, but the API permits the state and no merchant would expect it.
EXPECTED BEHAVIOUR	Validate that a bookable product belongs to at least one branch, or document that an empty array means "all branches".
ACCEPTANCE CRITERIA	Either the state is prevented, or its meaning is documented.



Performance

Measured, not estimated.

REK-042 **HIGH** All endpoints

Response times vary by nearly an order of magnitude on the same endpoint

DOCUMENTED	No performance characteristics documented.
ACTUAL	Measured sequentially from Frankfurt: <code>/branches</code> 0.8s to 3.3s, <code>/products</code> 1.2s to 10.8s, <code>/subscriptions</code> 1.5s to 3.4s, <code>/reservations</code> (100 rows) around 6s.
EVIDENCE	Repeated sequential measurement over a working day.
IMPACT	A ninefold spread on the same endpoint makes timeout selection guesswork. We settled on a 10s client timeout, which <code>/products</code> sometimes exceeds, and every such timeout on the booking path forces our reconciliation logic to run because we cannot tell whether the write landed (REK-004). Slow reads also mean our admin dashboard took up to 7.8s to assemble before we introduced caching.
EXPECTED BEHAVIOUR	Publish expected latency ranges, and investigate the tail on <code>/products</code> , which is the endpoint a booking flow must call first.
ACCEPTANCE CRITERIA	p95 latency is documented per endpoint and the <code>/products</code> tail is brought within it.

REK-043 **HIGH** All endpoints

Concurrent requests degrade catastrophically with no rate-limit signal

DOCUMENTED	Not documented.
ACTUAL	Firing six page requests concurrently made <code>/subscriptions</code> hang past two minutes, having answered in 1.5s moments earlier when called sequentially.
EVIDENCE	Direct comparison of sequential and concurrent pagination against the live tenant.
IMPACT	The instinctive optimisation (parallelise pagination) makes things dramatically worse, and there is no 429 or Retry-After to explain why, so an integrator concludes the API is unreliable rather than that they are being throttled. We serialise everything as a result, which makes REK-037 more painful. There is also a second-order risk: this same API serves the storefront where real customers check out, so a merchant's misjudged parallelism is load pointed at their own revenue path.
EXPECTED BEHAVIOUR	Either handle concurrency gracefully, or document a concurrency limit and enforce it with a clear 429 (see REK-038).
ACCEPTANCE CRITERIA	Concurrent requests either succeed within documented latency or are refused with 429 and Retry-After.



Credentials and access scope

Security posture that affects every merchant, not only us.

REK-044 HIGH Credentials

Only one credential scope exists, and it can read the entire customer list

DOCUMENTED	No scoping model is documented.
ACTUAL	<code>GET /customers</code> with our merchant key returns all 284 customer records, including names, mobile numbers and emails.
EVIDENCE	Single authenticated request against the live tenant.
IMPACT	A booking integration must hold this key on its server in order to create reservations. It therefore necessarily holds a credential that can read every customer record, which under PDPL is a significant amount of personal data. The blast radius of a leaked key is the entire customer base, for every merchant, with no way to reduce it. A booking integration needs to create reservations and read the catalog and slots; it does not need the customer list.
EXPECTED BEHAVIOUR	Scoped API keys. At minimum a catalog-and-booking scope that cannot read <code>/customers</code> .
ACCEPTANCE CRITERIA	A merchant can issue a key that successfully creates reservations and receives 403 on <code>GET /customers</code> .

REK-045 MEDIUM Credentials

A key is displayed once and cannot be re-read, with no dual-key rotation

DOCUMENTED	Not addressed.
ACTUAL	Keys can only be regenerated, and only one appears to be active at a time.
EVIDENCE	Dashboard behaviour at platform.rekaz.io.
IMPACT	Showing a secret once is good practice. Combined with REK-044 it becomes a problem: the recovery path for "we are not certain whether this key leaked" is a regeneration that breaks every live integration simultaneously, with no overlap window. A merchant facing a possible leak must therefore choose between an outage and continuing to run on a possibly-compromised credential.
EXPECTED BEHAVIOUR	Support two concurrently active keys per merchant so rotation can be done with overlap.
ACCEPTANCE CRITERIA	A merchant can create a second key, deploy it, and revoke the first without any request failing.

Open questions

Sent in our July 2026 review letter. Several findings above exist only because we never got an answer, so we answered some ourselves by testing against production, which is the only environment available (REK-030).

#	QUESTION	RELATED
Q1	Can the hosted payment page be branded with the merchant's identity, or served from a merchant subdomain (for example pay.mazj.org)?	REK-027
Q2	Is there any way to mark an invoice paid from outside Rekaz, so a merchant can collect through their own Saudi gateway while Rekaz remains the invoicing and ZATCA authority?	REK-026
Q3	Is there a sandbox or test tenant, or does all development run against production data?	REK-030
Q4	Are webhook payloads signed? If so, which header and what is the verification procedure? If not, what do you recommend for verifying origin?	REK-005
Q5	Are idempotency keys supported on POST /reservations/bulk?	REK-004
Q6	Do unpaid Pending reservations expire automatically and release their slot? After how long?	REK-006
Q7	What are the actual rate limits on the public API?	REK-038
Q8	What is the precise meaning of the slot fields, especially the relationship between isAvailable and availableReservationsCount, and the nature of the interval between from and to?	REK-014
Q9	Is there any end-customer authentication mechanism that would let a merchant build a "my bookings" page without using the admin key?	REK-028
Q10	Is there a refund endpoint, or is refunding dashboard-only?	REK-035
Q11	Is there an OpenAPI or Swagger file that can be relied on for code generation?	REK-032

What we answered ourselves, and what it cost. Q5 (idempotency) and Q8 (slot semantics) we settled by measurement: there is no idempotency mechanism, and across 192 real slots the contradictory state in your documented example never occurs. Q2, Q6 and the identity question behind REK-046 we could only answer by breaking our own site: we shipped a change on 2026-07-28 that always sent the submitted customer details, and discovered from the resulting outage that Rekaz refuses that payload with 403 for any mobile number it already holds. That is an expensive way to learn an API's rules, and a sandbox (Q3, REK-030) would have made it a five-minute test instead.



What works well

Stated for balance, and because it is true. This report exists because the model underneath is worth extending, not replacing.

- `GET /customers?mobileNumber=` filters correctly and quickly.
- `customerId` genuinely filters `GET /subscriptions`, narrowing 98 rows to 1.
- `POST /reservations/bulk` with inline `customerDetails` creates the customer and the reservation in one call, which is a genuinely good design for a first-time buyer. ⚠️ It works only for a mobile number Rekaz has not seen before; for a returning customer the same call is refused (REK-046).
- `PUT /reservations/{id}/cancel` returns 204 and genuinely releases the slot. We confirmed the window reappeared as available immediately.
- `skipCount` and `maxResultCount` paginate correctly and return disjoint pages, which is more than can be said for the filters on the same endpoint. ⚠️ A request above 100 is silently capped at 100 rather than refused (REK-037), the same silent-parameter pattern as REK-002.
- The slot model (overlapping windows sliding at one-hour granularity, sized by the price duration) is a good design once understood.
- `traceId` is genuinely useful where it appears, and we log it. ⚠️ It is present only in the ProblemDetails (400) shape; the 403, 404 and 500 bodies carry none, so the failures most likely to need support are the ones without a trace id (REK-012).
- The catalog, availability, reservation, subscription and package model covers our business without forcing us to rebuild an operations system. That is the reason we are writing this report rather than leaving.

The one sentence version. The catalog, availability, reservation and subscription model let us build a first-party booking experience without rebuilding an operations system, which is a genuinely strong foundation. What stopped us finishing it is the payment boundary (REK-025, REK-026), and what cost us most in getting there is that several endpoints fail silently rather than loudly (REK-002, REK-008, REK-015).

We are happy to provide request and response captures for any finding, to re-run any measurement, or to meet if that is faster. Every item above is reproducible against our tenant on request.